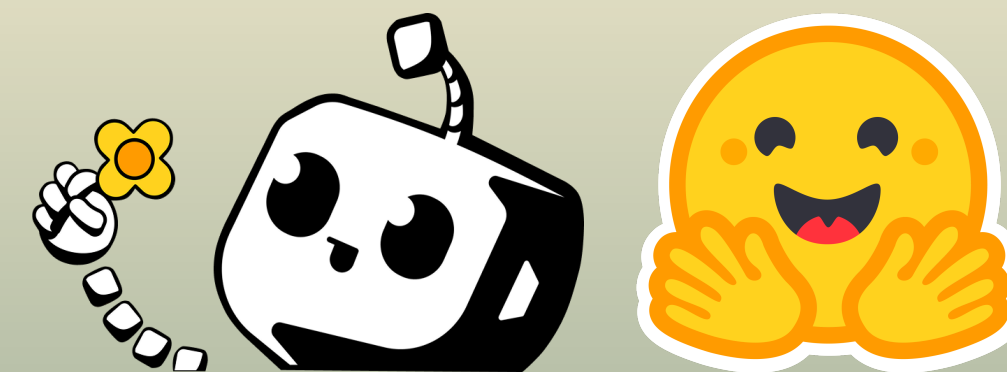


# GOSIM Paris 2026

May 5-6, 2026 Station F, Paris



## LeRobotDataset: Scaling Across Modalities

Caroline Pascal · Robotics Engineer, LeRobot · Hugging Face



HELLO

# A PhD in acoustics in an embodied AI team

**Caroline Pascal** — Robotics Engineer at **Hugging Face**, working on **LeRobot** library.

★ **datasets**

★ **hardware**

policies

teleoperation

tooling

## TODAY

Strong focus on our **dataset format**, making it scalable in **all dimensions**.



PART 01

# What is LeRobot?

A 90-seconds tour of an all-rounder robotics library.



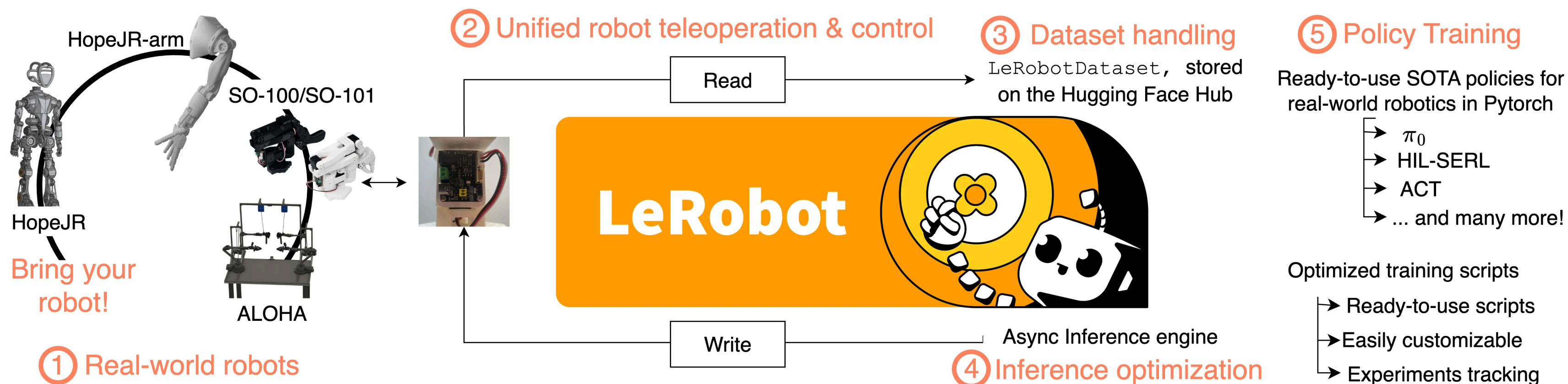
# Open-source robotics, the Hugging Face way

## OUR OBJECTIVE

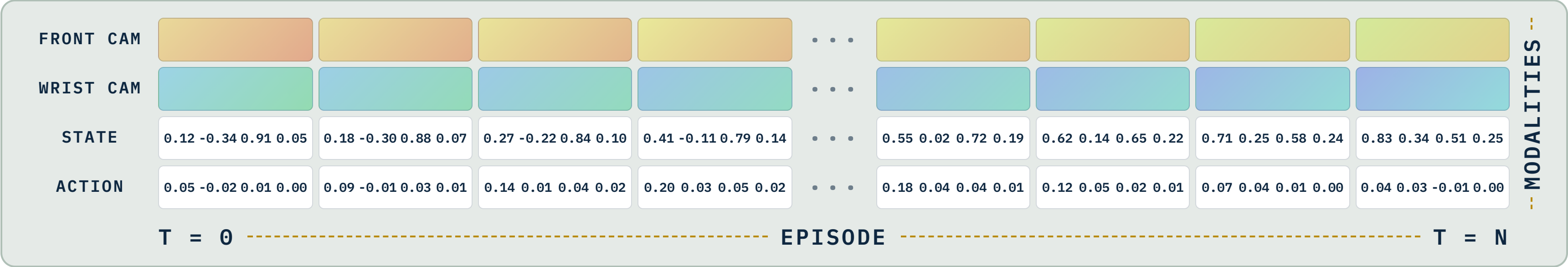
Make robotics accessible, based on **open-source software** and cheap hardware.

One library to **record** robots, **train** policies, **share** both on the Hub, and **reproduce** what others built.

As simple as `transformers.from_pretrained()`



# The robotics datasets challenges



**MORE EPISODES →**

**Storage ↔ Access**

Scale to **millions of episodes** without struggling to access single frames.

|                          |                        |
|--------------------------|------------------------|
| <b>DROID</b>             | 1.7 TB · 76k+ episodes |
| <b>AgiBot World Beta</b> | 48.1 TB · 1M+ episodes |
| <b>10Kh-Real0min</b>     | 36.9 TB · 5M+ episodes |

**MORE MODALITIES ↓**

**Coverage ↔ Specificity**

Each modality has its own **encoding, compression** and **access pattern**.

|                        |              |
|------------------------|--------------|
| <b>Yaak-AI</b>         | GPS + maps   |
| <b>Daimon Infinity</b> | tactile      |
| <b>BEHAVIOR-1K</b>     | depth frames |

PART 02

# The LeRobotDataset format

Built to scale: millions of episodes, terabytes of multi-modal data.



# Anatomy of a LeRobotDataset

## TABULAR

### Parquet

Actions, states, timestamps, index.  
Columnar, queryable, streamable.

## VISUAL

### MP4

One video file per camera.  
Random-access frames via keyframes.

## METADATA

### JSON

Schemas, episode boundaries, statistics.  
Self-describing.

```
my_dataset/
├── data/                                # tabular: actions, states, timestamps
│   ├── chunk-000/file-000.parquet      # many episodes per file
├── videos/                             # one MP4 shard per camera
│   ├── observation.images.front/chunk-000/file-000.mp4
│   └── observation.images.wrist/chunk-000/file-000.mp4
└── meta/
    ├── episodes/chunk-000/file-000.parquet # per-episode index, lengths, offsets
    ├── tasks.parquet                    # task definitions
    ├── stats.json                       # aggregated statistics
    └── info.json                        # schema, fps, features
```

# One file, many episodes

With **one episode per file**, the HF Hub was struggling to handle the scale.

V2.1 • 1 FILE / EPISODE

**~1M files**

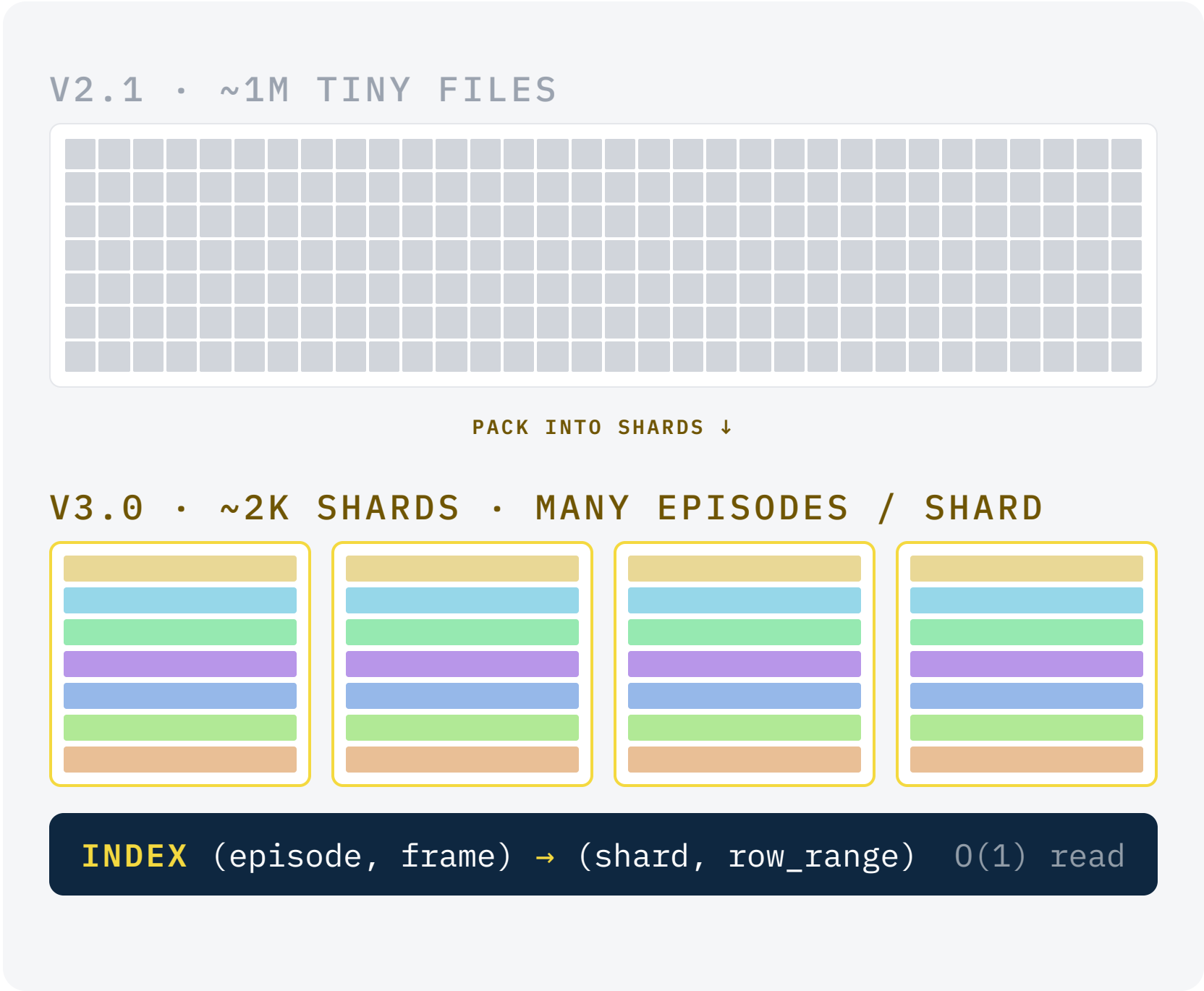
slow listing • slow uploads • slow random access

V3.0 • MANY EPISODES / FILE

**~2k shards**

chunked Parquet • memory-mapped • streamable

Episodes are concatenated into bounded **shards**. An index maps any (episode, frame) back to its row range.



# One MP4 file, many images

A 10-MINUTE EPISODE • 2 CAMERAS • 30 HZ

~36k

FRAMES PER CAMERA

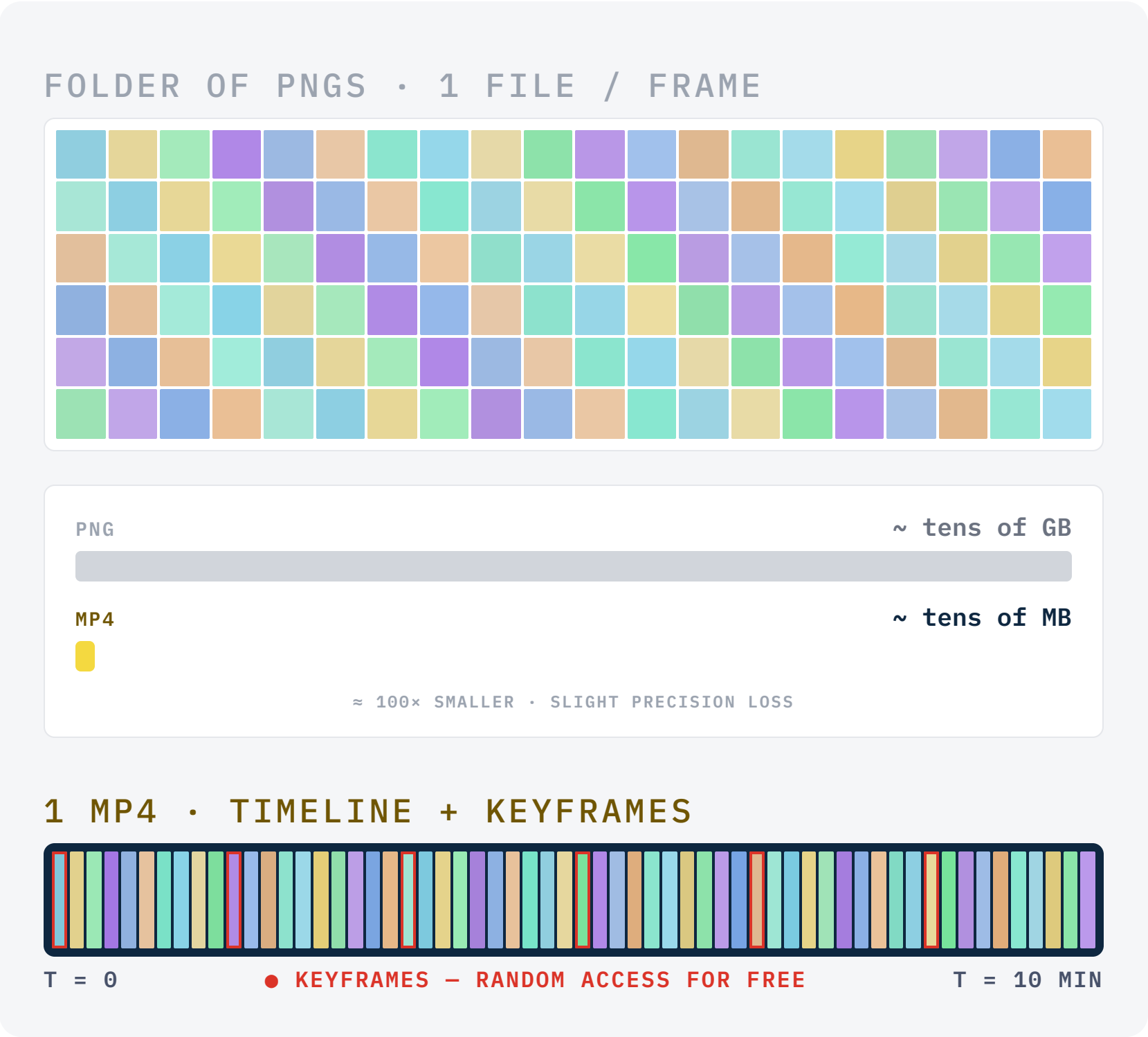
AS PNGS

~ tens of GB

AS MP4

~ tens of MB

Two orders of magnitude smaller - frame-accurate random access **for free** - **slight loss** in pixel precision.



PART 03

# Beyond RGB: depth frames

What happens when the camera no longer outputs colors.



# Depth maps: frames, but not really

RGB camera

## 3 × uint8

3 channels. RGB values 0–255. Suited for most video codecs.



Behavior 1K • simulated RGB camera

Depth camera

## 1 × uint16

1 channel. Millimeter values 0–65,535. Each bit matters.



Behavior 1K • same scene, depth (viridis)

# The naive workarounds

Naive · A

## Byte-pack into RGB

Split `uint16` across channels.

R = high byte

G = low byte

B = 0 (unused)

### Codec mangles low byte

Chroma compression kills millimeter precision.

Naive · B

## Apply colormap → encode

Map depth → viridis/turbo, save as RGB MP4 like a regular video.

`uint16`

→

RGB·8

→

.mp4

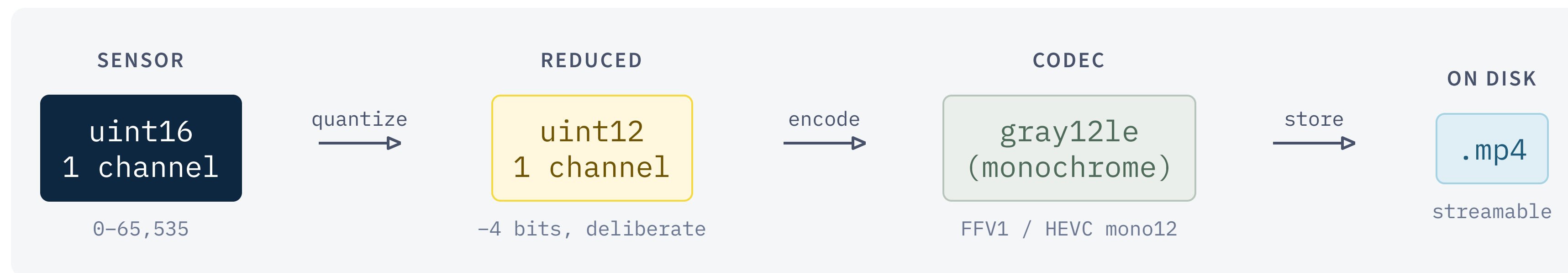
### Depth values are gone

Colormap is a lossy 1 → 3 mapping.

## THE COMMON CATCH

Both approaches treat depth like a picture. Depth is a **measurement**: every byte you drop is a millimeter you can't get back.

# Our pipeline: quantization + 12-bit grayscale encoding



## WHY UINT12

Highest bit-depth available MP4 encoding.

## WHY GRAYSCALE

One channel in, one channel out. No chrominance subsampling.

## THE TRADE

**+ One-channel grayscale MP4**  
**- Coarser depth resolution**

# Benchmark: smaller files, cleaner depth

so101\_depth\_raw · 172 frames · 480 × 640 · raw uint16 = 211 MB · 10 m dynamic range

| METHOD                                   | FILE SIZE      | MSE ↓ (MM <sup>2</sup> ) | PSNR ↑ (DB) | DECODE (MS/FRAME) |
|------------------------------------------|----------------|--------------------------|-------------|-------------------|
| TIFF (lossless ref.)                     | 56.1 MB        | 0                        | ∞           | 4.63              |
| Naive RGB pack + h264                    | 1.74 MB        | 704,703                  | 21.5        | 0.51              |
| <b>uint12 + grayscale HEVC (ours)</b>    | <b>0.72 MB</b> | <b>2,418</b>             | <b>46.2</b> | <b>0.60</b>       |
| uint12 + grayscale FFV1 (ours, lossless) | 9.04 MB        | 26                       | 65.9        | 3.39              |

**TAKEAWAY** → Same size as the hack, **+26 dB** closer to truth — and **~70×** smaller than lossless.

PART 04

# Listening: audio

A new modality: small samples, high frequencies.

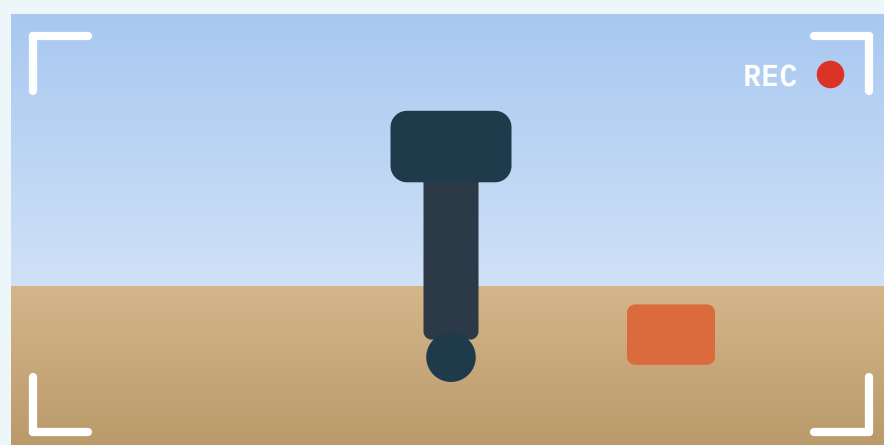


# A frequency mismatch

## Cameras

**30 Hz**

3 channels · uint8 · ~ MB / sec per camera

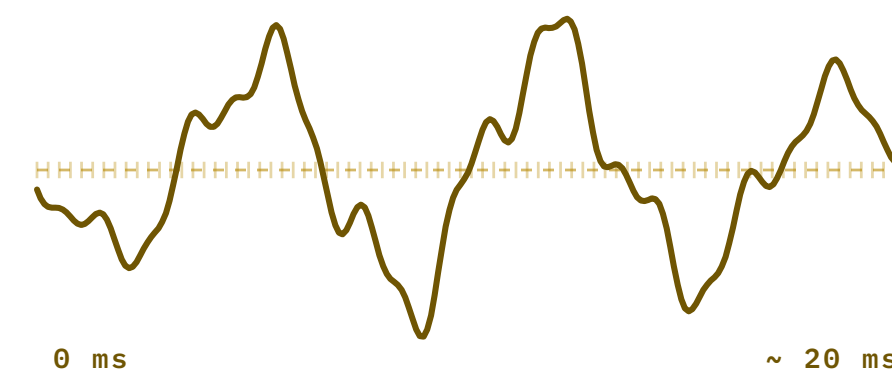


High bandwidth · low frequency

## Microphones

**48 kHz**

1 float32 / sample · ~ kB / sec



Low bandwidth · high frequency

## IMPLICATION

Audio cannot ride the camera loop. We need a separate acquisition path.

# Cameras spike, audio streams

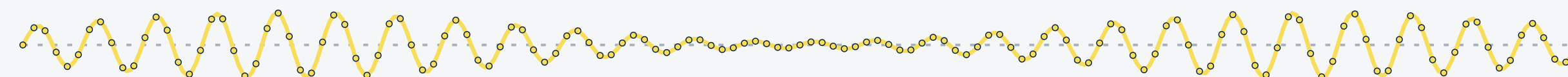
## CAMERAS

1 frame / 33 ms



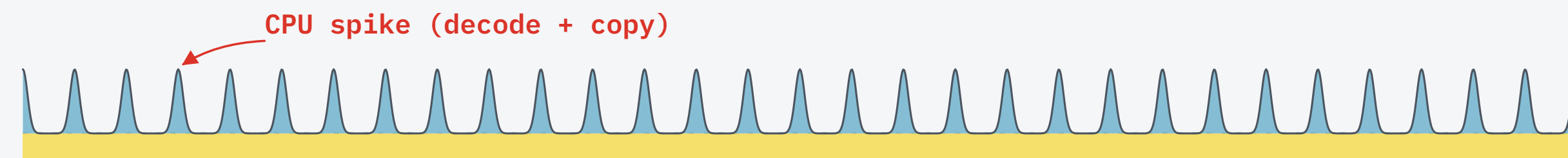
## MICROPHONES

48 000 samples / s



## CPU LOAD

aggregate  
both processes



0.00s

0.25s

0.50s

0.75s

1.00s

### CONTINUOUS, NOT FRAMED

Audio writes every sample over the whole recording.

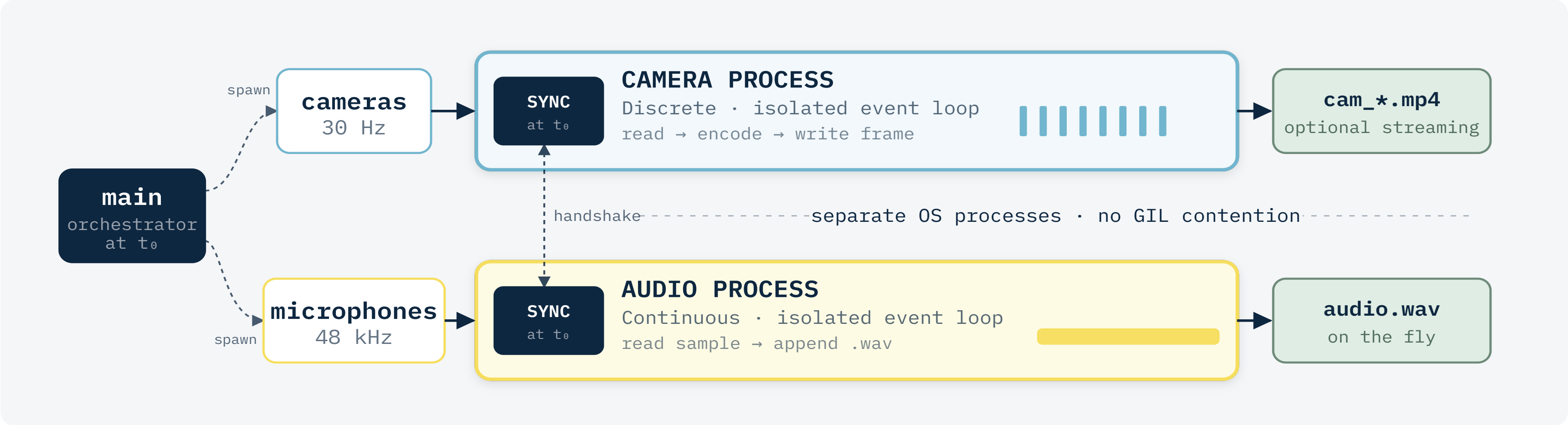
### STABLE CPU LOAD

Cameras generate CPU spikes at 30 Hz, but microphones require stability.

### SAME TIMELINE

Audio samples need to be synchronized with video frames.

# Recording: audio in its own process



## PROCESSES, NOT THREADS

Each lane gets its own interpreter and CPU core: true parallelism.

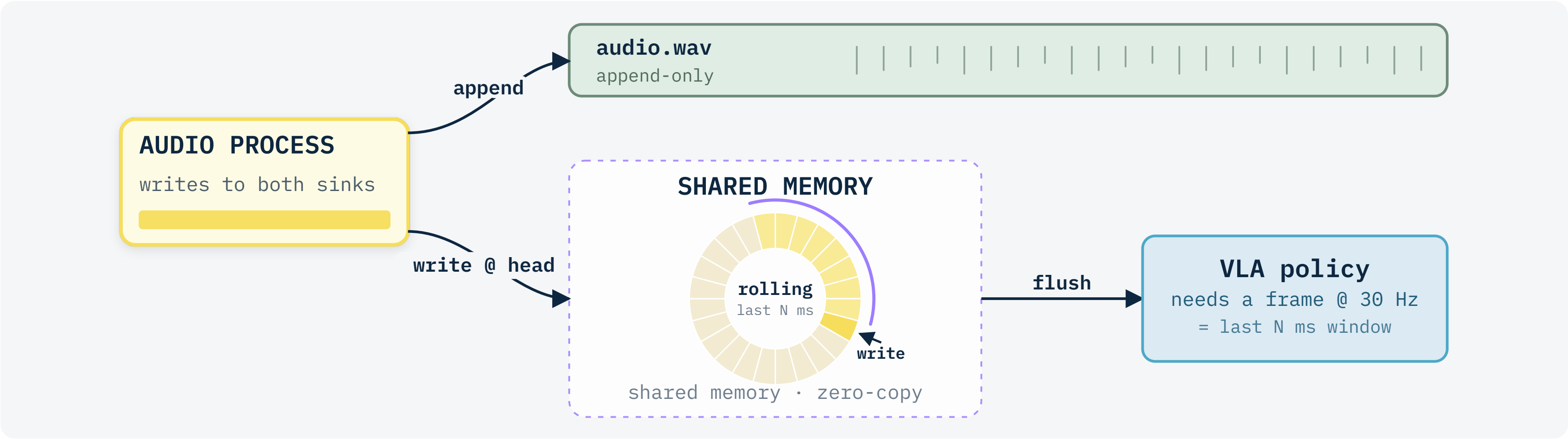
## SPAWN + SYNC AT $T_0$

One handshake aligns both lanes, then each runs on its own clock.

## WAV WRITE ON THE FLY

Samples appended straight into `audio.wav`. No batching.

# Inference: rolling buffer in shared memory



## FRAME AT 30 HZ

VLA expects a fixed-shape tensor but audio streams continuously.

## ROLLING BUFFER

Circular array: always holds the freshest N ms.

## ZERO-COPY SLICE

`multiprocessing.shared_memory =`  
no copy across processes.

PART 05

# What's next



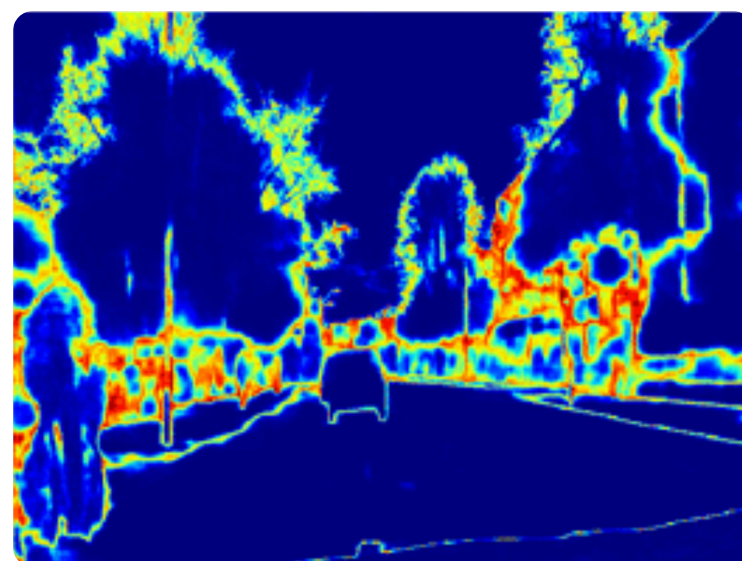
# Depth opens the door for image-space sensors

Once you have **quantize** → **grayscale codec** in your toolbox, anything that lives in **image-space** but isn't RGB just plugs in.



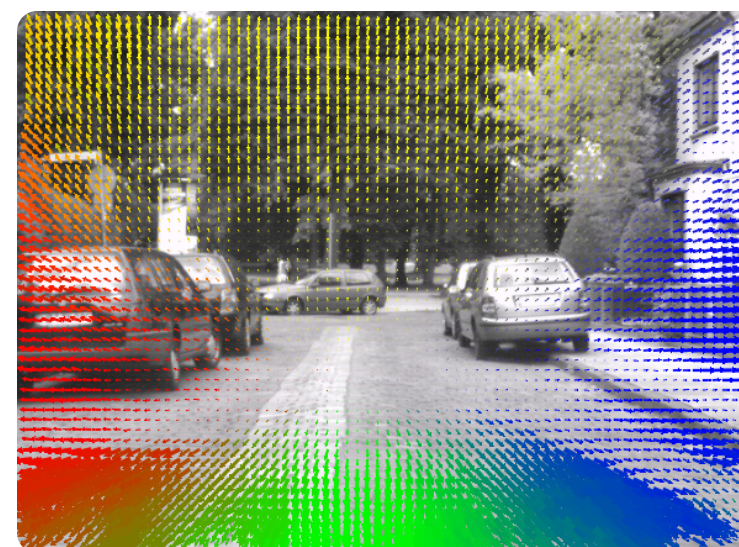
thermal

Single-channel infrared frames



confidence

Stereo-depth confidence maps



optical flow

2-channel motion fields

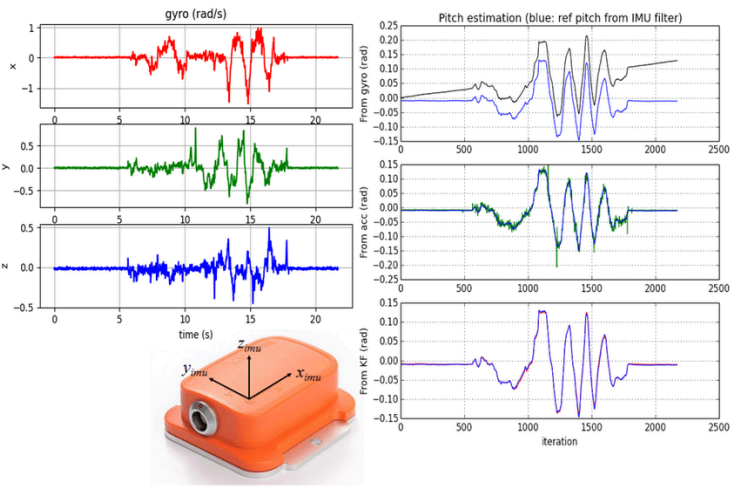


segmentation

Categorical masks

# Audio opens the door for high-frequency signals

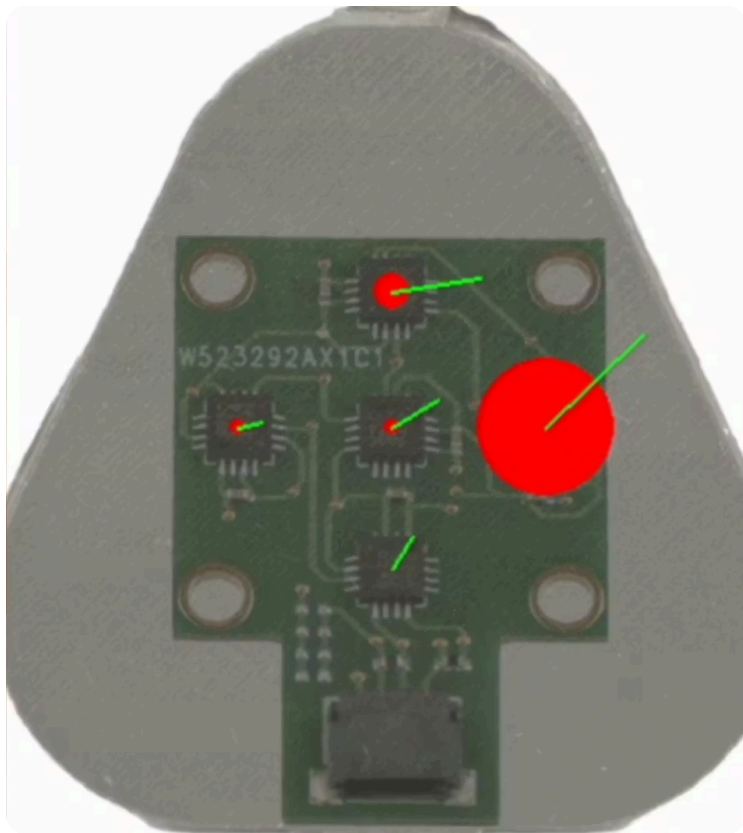
Once you have **multiprocessing + shared-memory ring**, any sensor faster than the cameras follows the same blueprint.



The image shows an orange IMU sensor with a coordinate system (x, y, z) and axes (x<sub>IMU</sub>, y<sub>IMU</sub>, z<sub>IMU</sub>). To its right are six plots: three for gyro (rad/s) in x, y, and z axes, and three for acceleration (rad) in x, y, and z axes. The plots show high-frequency signals over time (0 to 2500 iterations).

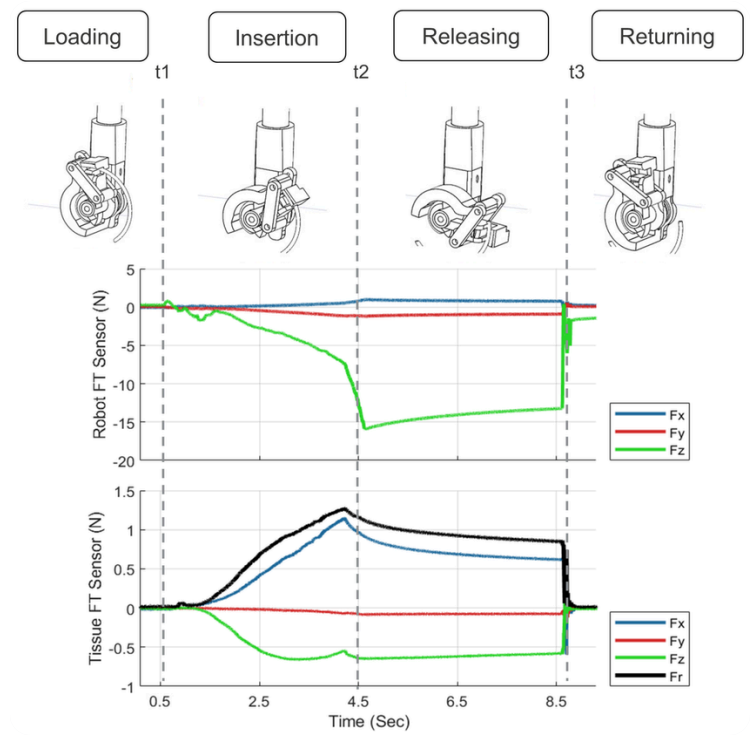
IMU

Accelerometers / gyros at 1 kHz



tactile sensing

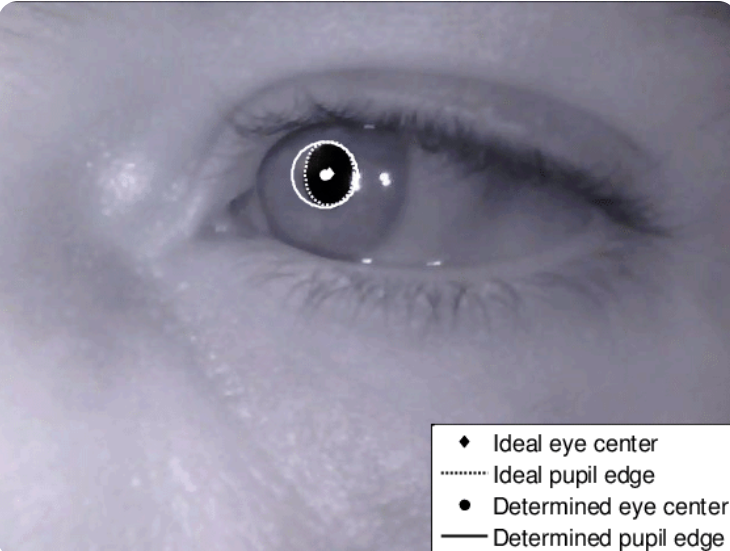
Skin sensors and taxels arrays



The image shows two line graphs and four diagrams. The top graph is labeled 'Robot FT Sensor (N)' and the bottom graph is labeled 'Tissue FT Sensor (N)'. Both graphs show force components (Fx, Fy, Fz) over time (0.5 to 8.5 seconds). The top graph has a legend for Fx (blue), Fy (red), and Fz (green). The bottom graph has a legend for Fx (blue), Fy (red), Fz (green), and Fr (black). The four diagrams show a robot arm in different states: Loading, Insertion, Releasing, and Returning. The x-axis is labeled 'Time (Sec)'.

force sensing

6-axis force and torque sensing



A photograph of a human eye with a black circle indicating the determined eye center. A legend in the bottom right corner shows: Ideal eye center (black dot), Ideal pupil edge (dotted line), Determined eye center (black dot), and Determined pupil edge (solid line).

eye tracking

Gaze samples at 100–250 Hz

# Two more on the near roadmap

in progress

## Text annotations

INPUT

USER

"Put the watermelon on the towel"

EMBODIED CHAIN-OF-THOUGHT

TASK

Place the watermelon on the towel

PLAN

1. Move to watermelon • 2. Firmly grasp • 3. Move to towel • 4. Place

SUBTASK REASONING

The watermelon is the first object the robot needs to interact with...

SUBTASK

Move to the watermelon

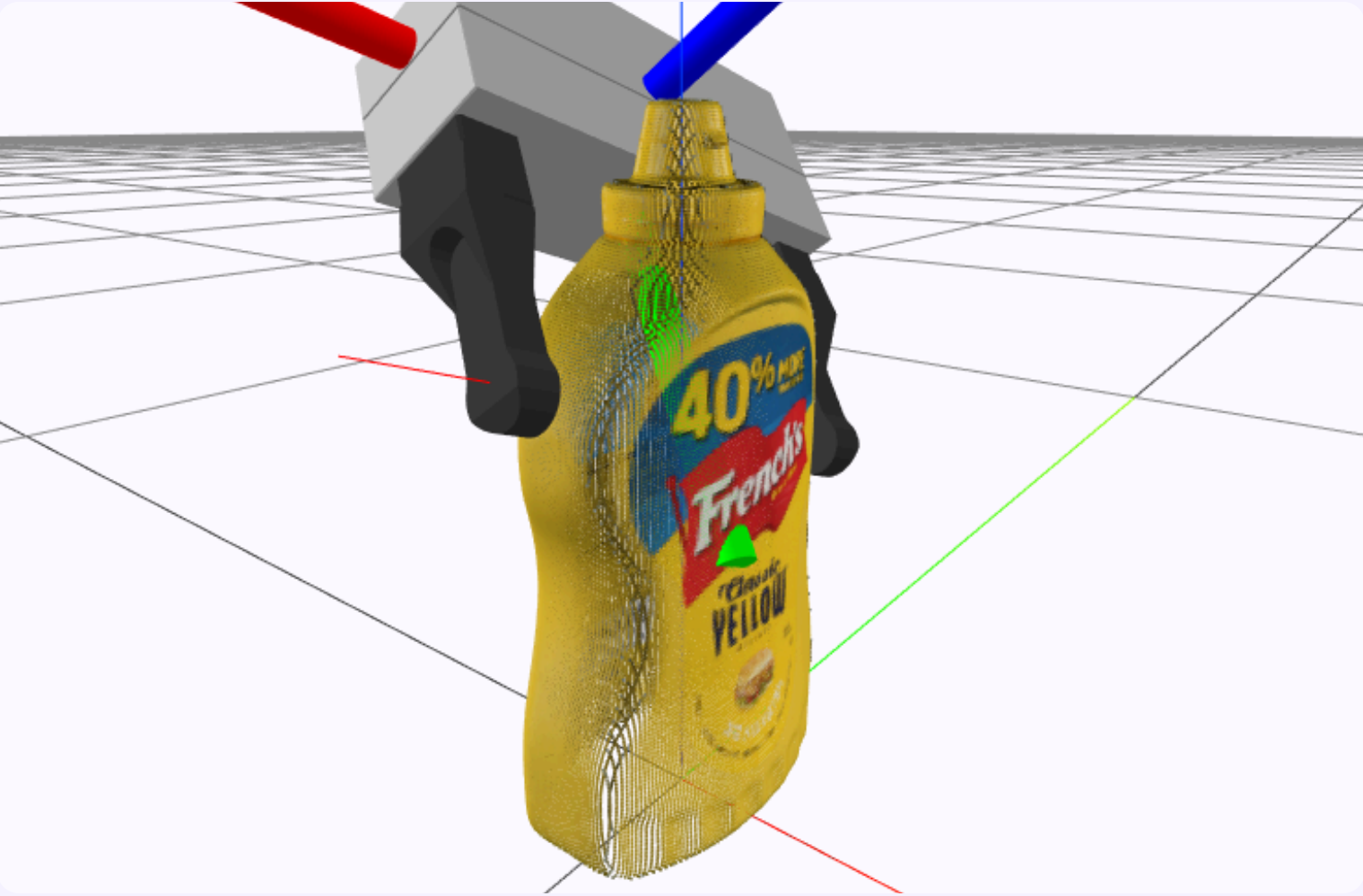
MOVE REASONING

The watermelon is behind the robot, so it needs to move backward

MOVE

in progress

## Point clouds



TRY & CONTRIBUTE

# Want to get a taste of it ?

```
$ pip install lerobot
```



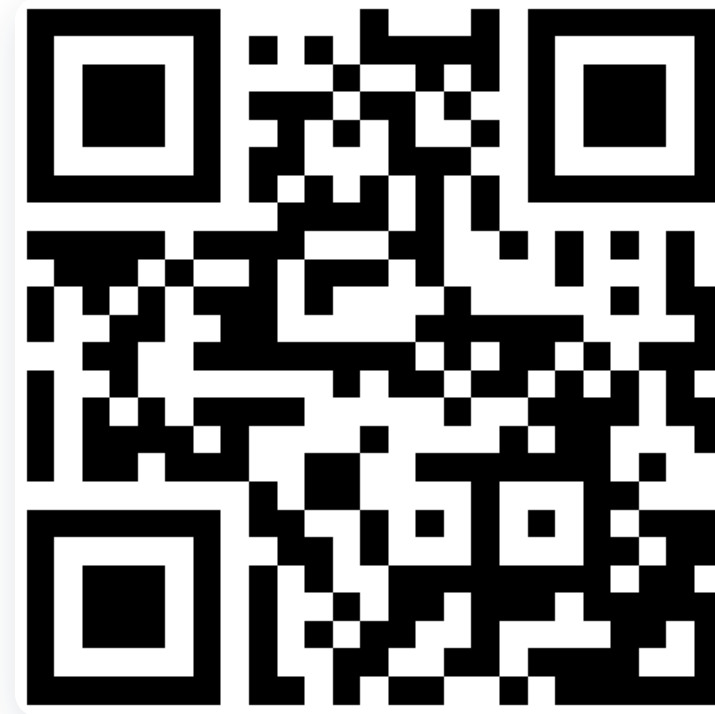
GITHUB



[github.com/huggingface/lerobot](https://github.com/huggingface/lerobot)



DISCORD



[discord.gg/q8Dzzpym3f](https://discord.gg/q8Dzzpym3f)

MERCI • THANK YOU

# Questions?

